

Decomposing Algebraic **Roots** into Low-Degree Sums and Products

Exact Mathematica algorithms, global lower bounds,
and the scope of minimality

A technical article with reference implementation

6 September 2026

Abstract

An algebraic number is easy to combine with other algebraic numbers, but reversing that process while minimizing the largest degree requires more than polynomial factorization. This article gives exact solutions to both degree-nine examples in the question and proves that their largest component degree, three, is globally optimal, even when arbitrarily many components and arbitrary algebraic extensions are allowed. It then develops three complementary algorithms: resultant-based candidate generation with exact branch certification; a complete enumeration of embedded subfields followed by rational linear algebra; and a normal-closure reduction that gives a finite algorithm for the unrestricted additive problem. For multiplication, an exact intersection criterion solves the two-factor problem in any specified number field. General finite products are also addressed through bounded exhaustive search, but are not confused with this two-factor problem. A worked example proves that restricting summands to the field generated by the input can change the answer from nine to six. The accompanying Wolfram Language package, tests, and independent exact verification scripts make the distinctions between a witness, a bounded negative result, and a global minimum explicit.

What is and is not delivered. Both requested examples are completely solved, with global optimality proofs. The additive theory is complete without a restriction on the number or location of summands. The supplied multiplicative field algorithm is complete for *at most two factors in a specified field*; its result is globally optimal whenever it meets the global lower bound. A universally terminating optimizer for arbitrary-length products in arbitrary algebraic extensions is *not* claimed by the implementation. The bounded product searches have explicitly bounded scopes. These are mathematical distinctions, not merely implementation details.

Contents

1	The problem and its conventions	3
2	The two requested examples: exact answers and optimality	3
2.1	Why the degrees really are nine	4
2.2	A lower bound that applies to arbitrarily many components	4
3	Resultants: generate candidates, then certify the chosen root	5
3.1	Two exact composition formulas	5
3.2	Why polynomial equality is too restrictive in general	6
3.3	Coefficient matching is a proposal mechanism	6
3.4	Certifying the actual conjugate	6
4	A finite, exact enumeration of all embedded subfields	7
4.1	Computing a coefficient field by rational linear algebra	8
4.2	What happens for the two degree-nine examples	8
5	All finite sums inside a specified number field	9
6	Two factors inside a specified field: an intersection test	9
6.1	Why two factors do not settle arbitrary finite products	10
7	A complete reduction for the unrestricted additive problem	11
7.1	Implementation and practical limits	11
8	A concrete warning: internal degree nine, global degree six	12
8.1	The internal answer is nine	12
8.2	An explicit global degree-six decomposition	12
8.3	Why six is globally minimal	13
9	What can be asserted for unrestricted products?	13
9.1	An exact bounded search for any number of factors	14
10	Using the supplied implementation	14
10.1	Validation status	15
10.2	Reproducibility	15
11	Conclusions	16
A	Complete Wolfram Language reference implementation	17

1 The problem and its conventions

Write

$$\text{adeg}(\beta) = [\mathbb{Q}(\beta) : \mathbb{Q}].$$

For an exact algebraic number α , define

$$D_+(\alpha) = \min_{\alpha = \beta_1 + \dots + \beta_r} \max_i \text{adeg}(\beta_i), \quad (1)$$

$$D_\times(\alpha) = \min_{\alpha = \beta_1 \dots \beta_r} \max_i \text{adeg}(\beta_i), \quad \alpha \neq 0. \quad (2)$$

Here r is any positive finite integer and the components lie in $\overline{\mathbb{Q}}$. For zero we set $D_\times(0) = 1$, witnessed by the single rational component zero. Rational components are allowed and have degree one. Requiring at least two components makes no essential difference: append zero to a sum or one to a product. Negative signs and inverses do not change absolute degree.

The coefficient field is crucial. Every displayed polynomial defining a component must have *rational* coefficients, or integer coefficients after clearing denominators. Allowing arbitrary algebraic coefficients would trivialize the problem: $X - \alpha$ has formal degree one over $\mathbb{Q}(\alpha)$ but does not make α a rational number. Likewise, a nested radical of small relative degree need not have small absolute degree.

The input polynomial may be reducible or nonminimal. The degree to optimize is therefore obtained from `MinimalPolynomial`, not by inspecting the syntax of the first argument of an arbitrary `Root`. Mathematica supports exact algebraic arithmetic through `Root`, `RootReduce`, and `MinimalPolynomial` [2, 3, 4].

There are three different scopes that must not be conflated:

Scope	Meaning
Unrestricted	Components may lie anywhere in $\overline{\mathbb{Q}}$; their number is unbounded.
Ambient-field	Every component must belong to a specified number field K .
Two-component	Only a sum or product of at most two components is considered.

A search with bounded coefficient height introduces a fourth restriction. An algorithm should report these restrictions with its result.

2 The two requested examples: exact answers and optimality

Set

$$p(X) = X^3 + X + 1, \quad q(X) = X^3 - X + 1,$$

and let a, b be their respective real roots:

```
a = Root[1 + # + #^3 &, 1];
b = Root[1 - # + #^3 &, 1];
```

The two polynomials in the question, written in descending order, are

$$P(X) = X^9 + 2X^7 - 3X^6 + X^5 - X^4 + 3X^3 - X - 1, \quad (3)$$

$$S(X) = X^9 + 6X^6 + 3X^5 - 15X^3 + 24X^2 - 4X + 8. \quad (4)$$

The answer is exactly

$$\alpha_{\times} = ab, \quad \alpha_{+} = a + b.$$

The following is an executable verification, not a numerical comparison:

```
alphaProduct = Root[
  -1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 + 2 #^7 + #^9 &, 1];
alphaSum = Root[
  8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 + 6 #^6 + #^9 &, 1];

{RootReduce[a b - alphaProduct],
 RootReduce[a + b - alphaSum]}
(* {0, 0} *)
```

For orientation only,

$$a \approx -0.682327803828019, \quad b \approx -1.324717957244746,$$

so

$$\alpha_{\times} \approx 0.903891894458348, \quad \alpha_{+} \approx -2.007045761072765.$$

Both P and S have exactly one real root. These root counts, their irreducibility, and the resultant identities below were independently checked with exact SymPy calculations in the supplied verification scripts.

2.1 Why the degrees really are nine

There is also a structural proof of irreducibility. The cubics p, q are irreducible by the rational-root test, and their discriminants are respectively -31 and -23 . Their splitting fields L_p, L_q have Galois groups S_3 . The only nontrivial proper Galois subfield of an S_3 -extension is its quadratic discriminant field. Those quadratic fields are different here; consequently

$$L_p \cap L_q = \mathbb{Q}, \quad \text{Gal}(L_p L_q / \mathbb{Q}) \simeq S_3 \times S_3.$$

These standard Galois-correspondence facts may be found in [10].

Let a_1, a_2, a_3 and b_1, b_2, b_3 be the roots. All nine pairs occur under the Galois action. If $a_i + b_j = a_k + b_\ell$, then $a_i - a_k = b_\ell - b_j$ belongs to $L_p \cap L_q = \mathbb{Q}$. Two conjugates cannot differ by a nonzero rational: iterate a finite-order Galois automorphism taking one to the other. Thus $i = k, j = \ell$.

For products, a repeated value would give $a_i/a_k = b_\ell/b_j \in \mathbb{Q}$. A rational ratio of nonzero conjugates is 1 or -1 , again by iterating an automorphism. Neither cubic has two opposite roots, since $p(X) + p(-X) = q(X) + q(-X) = 2$. Thus the nine products are also distinct. Both Galois orbits therefore have size nine, proving

$$\text{adeg}(\alpha_{+}) = \text{adeg}(\alpha_{\times}) = 9.$$

2.2 A lower bound that applies to arbitrarily many components

Theorem 2.1 (Prime-divisor lower bound). *Let $n = \text{adeg}(\alpha) > 1$, and let ℓ be the largest prime divisor of n . Then*

$$D_{+}(\alpha) \geq \ell, \quad D_{\times}(\alpha) \geq \ell.$$

The same obstruction applies to any rational expression in finitely many algebraic numbers of degree less than ℓ .

Proof. Suppose all components have degree at most d . The splitting field of a component of degree $m \leq d$ has Galois group embedded in S_m , so every prime dividing its degree is at most d . The Galois group of the compositum of these splitting fields embeds in their direct product. Its order therefore has no prime divisor greater than d . The degree of any subfield, in particular $\mathbb{Q}(\alpha)$, divides that order. Hence every prime divisor of n is at most d . $\square$

For $n = 9$ this excludes all decompositions into linear and quadratic algebraic numbers, regardless of their number. The cubic witnesses above attain the bound. Therefore

$$D_{\times}(\alpha_{\times}) = 3, \quad D_{+}(\alpha_{+}) = 3.$$

In this special case the argument can be shortened: any finite collection of quadratic numbers lies in a multiquadratic extension of degree a power of two; its subfields cannot have degree nine.

3 Resultants: generate candidates, then certify the chosen root

The original Mathematica discussion contains a companion-matrix construction by yarchik and a resultant construction by Daniel Lichtblau [1]. These are useful ways to recover a prescribed shape of answer. The important additional steps are to retain rational coefficient fields, allow the correct polynomial divisibility condition, identify the requested conjugate, and prove minimality separately.

3.1 Two exact composition formulas

For monic $f, g \in \mathbb{Q}[X]$ of degrees m, n , define

$$C_{+}(f, g)(X) = \text{Res}_Y(f(Y), g(X - Y)), \quad (5)$$

$$C_{\times}(f, g)(X) = \text{Res}_Y(f(Y), Y^n g(X/Y)). \quad (6)$$

If f has roots u_i and g has roots v_j , these are

$$C_{+}(f, g) = \prod_{i,j} (X - u_i - v_j), \quad C_{\times}(f, g) = \prod_{i,j} (X - u_i v_j).$$

These identities follow immediately by evaluating the second polynomial at all roots of the first. The second expression is a polynomial: if $g(X) = \sum_{j=0}^n g_j X^j$, use

$$Y^n g(X/Y) = \sum_{j=0}^n g_j X^j Y^{n-j}$$

rather than performing a rational substitution followed by unsafe cancellation. This formula also handles zero roots correctly. Mathematica's **Resultant** implements the required elimination [5].

For the two cubics,

$$C_{\times}(p, q) = P, \quad C_{+}(p, q) = S.$$

Using the package in the archive:

```
Get["RootDecomposition.wl"];
ComposedPolynomial[x^3 + x + 1, x^3 - x + 1, x, "Product"]
ComposedPolynomial[x^3 + x + 1, x^3 - x + 1, x, "Sum"]
```

The implementation normalizes the input polynomials to monic form. The resultants then already have leading coefficient one. There is no need for an unexplained sign correction to a characteristic polynomial.

3.2 Why polynomial equality is too restrictive in general

Let M_α be the minimal polynomial of the input. A necessary and sufficient condition for *some* pair of roots of f, g to produce α is

$$M_\alpha \mid C_\circ(f, g), \quad \circ \in \{+, \times\}.$$

When $mn = \text{adeg}(\alpha)$, monicity makes divisibility equivalent to equality. Without that degree equality the composed polynomial may be reducible or have repeated roots.

For example, take $f = X^2 - 2$ and $g = X^2 - 8$. One sum of their roots is $\sqrt{2}$, using $-\sqrt{2} + \sqrt{8} = \sqrt{2}$. The composed polynomial is

$$(X^2 - 2)(X^2 - 18),$$

not the minimal polynomial $X^2 - 2$. Searching only by equating a composed polynomial to the input would miss this valid pair.

Nor is $\text{adeg}(\alpha)$ generally a divisor of the product of component degrees. If $c = \sqrt[3]{2}$ and $\omega = e^{2\pi i/3}$, then

$$c + 2\omega c = i\sqrt{3} \sqrt[3]{2}$$

has minimal polynomial $X^6 + 108$, although each summand has degree three. The degree of the compositum is six, not nine. The safe universal statement is an *inequality* $\text{adeg}(\alpha) \leq \prod_i \text{adeg}(\beta_i)$; stronger claims need additional hypotheses.

3.3 Coefficient matching is a proposal mechanism

For proposed degrees m, n , introduce generic monic rational polynomials f, g and set the coefficients of

$$C_\circ(f, g) \bmod M_\alpha$$

to zero. If $mn = \deg M_\alpha$, one may instead equate all coefficients of $C_\circ(f, g)$ and M_α . This produces a polynomial system in the unknown coefficients. Solving it over $\mathbb{C}$ or $\mathbb{R}$ is not the same as solving it over $\mathbb{Q}$. An algebraic solution for the coefficients can violate the original objective. A parameterized answer from `Solve` also needs an actual admissible specialization and subsequent exact checks [9].

Translations $u \mapsto u + r$, $v \mapsto v - r$ with $r \in \mathbb{Q}$ preserve a sum. Rescalings $u \mapsto ru$, $v \mapsto v/r$ with $r \in \mathbb{Q}^\times$ preserve a product. They can help reduce a symbolic search, but multiplicative gauges require care: setting the constant coefficient of a monic degree- m polynomial to one by rational rescaling requires a suitable rational m th root. That is not always possible. The package avoids assuming such a gauge.

3.4 Certifying the actual conjugate

A minimal polynomial does not specify which conjugate was intended. An exact identity is checked with

```
RootReduce[candidate - alpha] == 0
```

not with numerical closeness. The package enumerates roots of the proposed polynomials and tests each appropriate pair:

```
productAnswer = PairFromPolynomials[
  alphaProduct, x^3 + x + 1, x^3 - x + 1, x, "Product"];
sumAnswer = PairFromPolynomials[
```

```

alphaSum, x^3 + x + 1, x^3 - x + 1, x, "Sum");

productAnswer["Components"]
(* {Root[1 + # + #^3 &, 1], Root[1 - # + #^3 &, 1]} *)
productAnswer["GlobalMinimumCertified"]
(* True *)

```

The displayed comments indicate mathematically expected results; the native Wolfram tests supplied with the archive were not executed in this environment.

A cheaper candidate search guesses only one polynomial. For each of its roots u , compute $v = \alpha - u$ or $v = \alpha/u$ and measure its actual degree. This avoids guessing the second polynomial altogether:

```

PairFromOnePolynomial[alphaProduct, x^3 + x + 1, x, "Product", 3]
BoundedTwoRootSearch[alphaProduct, "Product", 3, 1]

```

The second call enumerates primitive integer first-component polynomials of degree at most three and coefficient height at most one. The polynomial $X^3 + X + 1$ lies in that search. Its complementary factor is not required to satisfy the same height bound. A failed bounded search proves only the corresponding bounded two-component statement, not unrestricted impossibility.

4 A finite, exact enumeration of all embedded subfields

The next method requires no guess about the factor polynomials. It is not intended to replace sophisticated subfield algorithms, but it is small enough to implement directly in Wolfram Language and to prove from first principles. More efficient approaches are discussed in [11, 12].

Let $K = \mathbb{Q}(\theta)$ have degree N , with monic minimal polynomial $M(T)$. Factor it over K :

$$M(T) = (T - \theta)g_1(T) \cdots g_s(T),$$

where all factors are monic and irreducible in $K[T]$. Characteristic zero ensures squarefreeness. Mathematica provides such factorization through `FactorList[... , Extension -> theta]` [6].

Theorem 4.1 (Subfields from coefficient fields of divisors). *Every intermediate field $\mathbb{Q} \subseteq E \subseteq K$ is generated over $\mathbb{Q}$ by the coefficients of a divisor*

$$h(T) = (T - \theta) \prod_{j \in J} g_j(T)$$

for some subset $J \subseteq \{1, \dots, s\}$. If h is the minimal polynomial of θ over E , then

$$[E : \mathbb{Q}] = N / \deg h.$$

Conversely, if the coefficient field of such an h has this degree, h is its minimal polynomial for θ .

Proof. Take the minimal polynomial h_E of θ over E . It divides M in $E[T]$, hence in $K[T]$, and contains the factor $T - \theta$. Its coefficients therefore arise from a divisor of the asserted form. Let F be the field generated by those coefficients. Then $F \subseteq E$, while $h_E(\theta) = 0$ gives

$$[K : F] \leq \deg h_E = [K : E].$$

The reverse inequality follows from $F \subseteq E$, so $F = E$. Conversely, if F is the coefficient field of h , the minimal polynomial of θ over F divides h . Equality of their degrees gives equality of these monic polynomials. $\square$

Thus the following finite procedure returns every embedded subfield. Enumerate the 2^s subsets J . Form h . Skip it unless $\deg h$ divides N . Compute the rational algebra generated by its coefficients. Retain it when its dimension is $N/\deg h$, and remove duplicate embedded fields.

4.1 Computing a coefficient field by rational linear algebra

Represent an element of K by its coordinate row in

$$1, \theta, \dots, \theta^{N-1}.$$

Products of coordinate rows are computed by multiplying the corresponding polynomials and reducing modulo M . Starting with the row for one, repeatedly adjoin products of the current basis rows with the coefficient generators, then row-reduce. The dimension can increase at most $N - 1$ times.

The final space is a finite-dimensional $\mathbb{Q}$ -subalgebra of the field K . It is itself a field: multiplication by a nonzero element is injective on that finite-dimensional algebra, hence surjective, so its inverse belongs to the algebra. This proves the closure computation correct.

Canonical reduced row spaces identify *embedded* subfields, not just isomorphism classes. Two isomorphic subfields in different positions in K can contribute differently to the desired decomposition and must not be merged.

The package first replaces a nonintegral generator by an integer multiple that is integral. This is an exact change of generator, not a change of field. It matters because `ToNumberField[b,theta]` guarantees the indicated `AlgebraicNumber[theta,...]` basis when θ is integral; it can otherwise normalize the generator [7, 8]. The code verifies the generator and the rational coordinates rather than silently assuming a basis.

4.2 What happens for the two degree-nine examples

For both inputs the irreducible factor degrees of M over K are

$$1, 2, 2, 4.$$

Only eight divisors containing the distinguished linear factor need be examined. The embedded subfield degrees returned are

$$1, 3, 3, 9.$$

The two degree-three subfields are $\mathbb{Q}(a)$ and $\mathbb{Q}(b)$. They are discovered from the degree-nine input alone; neither cubic is supplied to the algorithm.

```
AllSubfieldBases[alphaProduct][\"Degrees\"]
(* {1, 3, 3, 9} *)
AllSubfieldBases[alphaSum][\"Degrees\"]
(* {1, 3, 3, 9} *)
```

The pure subset enumeration is exponential in the number of factors. In a Galois ambient field of degree N , the minimal polynomial of a primitive element splits into N linear factors, making the count 2^{N-1} . The default `\"MaxDivisors\"` guard returns an explicit resource failure; it never converts an incomplete enumeration into a negative mathematical answer.

5 All finite sums inside a specified number field

For a finite extension $K/\mathbb{Q}$ and a degree bound d , define

$$V_d(K) = \sum_{\substack{\mathbb{Q} \subset E \subset K \\ [E:\mathbb{Q}] \leq d}} E,$$

where the sum is a sum of $\mathbb{Q}$ -vector subspaces, not a compositum of fields.

Theorem 5.1 (Additive membership criterion). *An element $\alpha \in K$ is a finite sum of elements of absolute degree at most d in K if and only if $\alpha \in V_d(K)$.*

Proof. A permitted summand β belongs to the subfield $\mathbb{Q}(\beta)$ of degree at most d , proving necessity. Conversely, each element of a degree-at-most- d subfield has absolute degree at most d . A vector-space expression in the sum of these fields is therefore a permitted additive decomposition. $\square$

Once subfield bases are known, stack their coordinate rows into a matrix B . The decision is the exact consistency test

$$B^T c = v_\alpha.$$

Solving this rational system gives the summands. All scalar coefficients are rational and can be absorbed into the summands without increasing their degrees. Grouping contributions from the same subfield cannot increase the maximum degree. Only the distinct subfield degrees need be tried, in increasing order.

This handles *any finite number of summands*; it is not a binary search. Indeed a basis selected from the union of all permitted field bases uses at most $[K : \mathbb{Q}]$ elements, so that many summands always suffice whenever membership holds.

The package also centers each nonrational summand by subtracting its mean trace

$$\mu(\beta) = \frac{\text{Tr}_{\mathbb{Q}(\beta)/\mathbb{Q}}(\beta)}{\text{adeg}(\beta)}.$$

It collects the removed rational quantities into one rational summand. This preserves the identity and the degree bound. For the requested sum example it removes arbitrary rational translations introduced by linear algebra and recovers the two depressed cubics.

```
s = MinSumInField[alphaSum];
s["MaxDegree"]           (* 3 *)
s["ExactIdentityVerified"] (* True *)
s["GlobalMinimumCertified"] (* True *)
```

The last statement is justified by Theorem 2.1, not merely by the fact that the internal search was exhaustive.

6 Two factors inside a specified field: an intersection test

Multiplication is not a linear operation, but its two-factor version has an especially simple linear reformulation.

Theorem 6.1 (Two-factor intersection criterion). *Let $E, F \subseteq K$ be subfields and $\alpha \in K^\times$. Then*

$$\alpha \in E^\times F^\times \iff E \cap \alpha F \neq \{0\}.$$

Given a nonzero element $u = \alpha v$ of that intersection, with $u \in E$ and $v \in F$, a factorization is $\alpha = u v^{-1}$.

Proof. If $\alpha = bc$ with $b \in E^\times, c \in F^\times$, then $b = \alpha c^{-1}$ lies in the intersection. Conversely, a nonzero $u = \alpha v$ gives $\alpha = u/v$, and v^{-1} belongs to F . $\square$

Let U and V be row bases for E and F . Multiply every row of V by α , producing the row matrix W . A nonzero rational null vector of

$$\begin{bmatrix} U^\top & -W^\top \end{bmatrix}$$

gives coefficients for u and v . Since U, V are bases and $\alpha \neq 0$, a nonzero null vector cannot give $u = v = 0$. Thus a nontrivial nullspace is exactly the desired condition. No norm equation, logarithm, or rational-point solver is needed for this two-field problem.

Enumerate all pairs of embedded subfields, sorted by $\max([E : \mathbb{Q}], [F : \mathbb{Q}])$, and apply this test. The first success minimizes the largest degree among all two-factor decompositions *inside the ambient field*. This remains true when a recovered factor generates a smaller field: that smaller subfield also occurs in the enumeration.

```
r = MinTwoFactorInField[alphaProduct];
r["MaxDegree"]           (* 3 *)
r["ComponentDegrees"]    (* {3, 3} *)
r["ExactIdentityVerified"] (* True *)
r["GlobalMinimumCertified"] (* True *)
```

A basis-dependent answer may be a rationally rescaled version of a, b . For example, the independent implementation produced $-2a/3$ and $-3b/2$, whose minimal polynomials are

$$X^3 + \frac{4}{9}X - \frac{8}{27}, \quad X^3 - \frac{9}{4}X - \frac{27}{8}.$$

Their product is exactly ab . The degree objective is unchanged by rational rescaling. The earlier `PairFromPolynomials` call gives the particular small-height representatives stated in the question.

6.1 Why two factors do not settle arbitrary finite products

Consider

$$\gamma = (1 + \sqrt{2})(1 + \sqrt{3})(1 + \sqrt{5}).$$

It has degree eight and is a product of three quadratic numbers. No product of only two quadratic numbers can have degree eight, since their compositum has degree at most four. Thus optimizing over two components can give a strictly worse answer than optimizing over all finite products.

In a fixed ambient field, the arbitrary-product membership problem is

$$\alpha \in H_d(K) := \langle E^\times : E \subseteq K, [E : \mathbb{Q}] \leq d \rangle.$$

There are finitely many such subfields, and factors from the same subfield can be combined. Nevertheless, testing a product of many multiplicative groups is not the same as testing one pairwise intersection. Greedy recursive splitting is a useful search strategy but does not by itself prove the global optimum.

7 A complete reduction for the unrestricted additive problem

The preceding additive algorithm becomes globally complete when its ambient field is the normal closure of the input field. The point is not just that this field contains many conjugates: normalized trace preserves the component degree bound.

Theorem 7.1 (Normal-closure descent for sums). *Let $L/\mathbb{Q}$ be finite Galois and contain α . For every $d \geq 1$,*

$$\alpha \text{ is a finite sum of numbers of degree at most } d \iff \alpha \in V_d(L).$$

Thus the unrestricted additive minimum is attained by a decomposition entirely inside the normal closure of $\mathbb{Q}(\alpha)$.

Proof. Suppose $\alpha = \sum_i \beta_i$ with $\text{adeg}(\beta_i) \leq d$. Choose a finite Galois extension $M/\mathbb{Q}$ containing L and all the β_i . Set $G = \text{Gal}(M/\mathbb{Q})$ and $N = \text{Gal}(M/L)$. Since $L/\mathbb{Q}$ is Galois, N is normal in G . Define the rational-linear averaging operator

$$\pi(z) = \frac{1}{|N|} \sum_{\sigma \in N} \sigma(z).$$

It fixes L , maps M to L , and commutes with the action of G . Therefore

$$\alpha = \sum_i \pi(\beta_i).$$

The map from the G -orbit of β_i to that of $\pi(\beta_i)$ is surjective and equivariant: $g\beta_i \mapsto \pi(g\beta_i) = g\pi(\beta_i)$. Orbit size is absolute algebraic degree, so

$$\text{adeg}(\pi(\beta_i)) \leq \text{adeg}(\beta_i) \leq d.$$

Zero summands may be omitted. This proves the forward direction. The reverse direction is Theorem 5.1 applied to L . $\square$

This proves a genuinely finite algorithm: construct the normal closure, enumerate its subfields, and use rational linear algebra at successively larger degree thresholds. There is no coefficient-height bound and no unbounded search over numbers of summands. Termination here is a mathematical statement, not a claim of acceptable running time on every input.

7.1 Implementation and practical limits

To construct the normal closure, list every exact root of the input's minimal polynomial and put them in their smallest common number field. The package uses `ToNumberField[roots, All]`; the `All` specification matters because the default common-field construction need not use the smallest common extension [7].

The public entry point is

```
MinSumGlobally[alphaSum]
```

It first searches inside $\mathbb{Q}(\alpha)$. If that witness meets the prime-divisor lower bound, global optimality is already established and no normal closure is constructed. This is what happens for the requested sum example.

Otherwise it constructs the normal closure and invokes the complete field procedure. The package has explicit degree and divisor-count guards. Removing those guards gives the finite exhaustive algorithm described by the proof:

```
MinSumGlobally[alpha,
  "MaxDivisors" -> Infinity, "MaxNormalDegree" -> Infinity]
```

This is not a performance recommendation. The normal closure can have degree as large as $n!$, and the elementary subfield enumeration can be much worse than the dedicated algorithms in [11, 12]. In particular, the degree guard is checked after the common-field construction; an outer `TimeConstrained` may be needed to bound that construction as well. A timeout or resource failure proves no nonexistence assertion.

8 A concrete warning: internal degree nine, global degree six

The same product input $\alpha_\times = ab$ gives a particularly instructive counterexample to silently restricting summands to $\mathbb{Q}(\alpha)$.

8.1 The internal answer is nine

The subfields of $K = \mathbb{Q}(ab) = \mathbb{Q}(a, b)$ have degrees 1, 3, 3, 9. The sum of its proper subfields is

$$\mathbb{Q}(a) + \mathbb{Q}(b) = \text{span}_{\mathbb{Q}}\{1, a, a^2, b, b^2\}.$$

The fields have compositum degree nine, so

$$\{a^i b^j : 0 \leq i, j \leq 2\}$$

is a basis of K . In particular ab does not lie in that five-dimensional sum. The internal additive minimum is therefore nine. The exact linear-algebra verification script obtains the same result.

8.2 An explicit global degree-six decomposition

Let $a_1 = a, b_1 = b$ and label each remaining pair by complex conjugation. Define

$$\begin{aligned} t_1 &= a_1 b_1 + a_2 b_2 + a_3 b_3, \\ t_2 &= a_1 b_1 + a_2 b_3 + a_3 b_2. \end{aligned}$$

Since each cubic has zero trace,

$$t_1 + t_2 = 2a_1 b_1 + (a_2 + a_3)(b_2 + b_3) = 3a_1 b_1.$$

The six bijective matching sums $\sum_i a_i b_{\sigma(i)}$, $\sigma \in S_3$, form a Galois-stable set. Their polynomial is

$$R(T) = T^6 + 6T^4 - 27T^3 + 9T^2 - 81T + 4.$$

One exact elimination certificate is

$$\begin{aligned} \text{Res}_U \left(U^3 + U + 1, \text{Res}_V \left(V^3 - V + 1, T^2 - 3UVT + 3U^2 - 3V^2 + 4 \right) \right) \\ = R(T)^3. \quad (7) \end{aligned}$$

Indeed $t_1 + t_2 = 3ab$ and $t_1 t_2 = 3a^2 - 3b^2 + 4$. The polynomial R is irreducible and has exactly two real roots, namely t_1, t_2 . Consequently

```

r1 = Root[4 - 81 # + 9 #^2 - 27 #^3 + 6 #^4 + #^6 &, 1];
r2 = Root[4 - 81 # + 9 #^2 - 27 #^3 + 6 #^4 + #^6 &, 2];
RootReduce[(r1 + r2)/3 - alphaProduct]
(* 0 *)

```

The summands $r_1/3, r_2/3$ each have degree six. They can individually be converted to rational-polynomial `Root` objects by `RootReduce`.

8.3 Why six is globally minimal

This stronger lower bound has a short group-theoretic proof. Work in $L = L_p L_q$ with $G = S_3 \times S_3$. Let A_1, A_2 be the normal cyclic subgroups of order three in the two factors, and let e_i be averaging over A_i . Put

$$T = (1 - e_1)(1 - e_2).$$

The zero traces of a, b give $T(ab) = ab \neq 0$.

Now let $\beta \in L$ have degree at most five and let H be its stabilizer. Its index divides 36, so it is 1, 2, 3, or 4. If the index is 1, 2, or 4, then H contains the unique Sylow three-subgroup $A_1 A_2$, so $T\beta = 0$. If the index is three, H has order twelve and $H \cap (A_1 A_2)$ has order three. Moreover H maps onto $G/(A_1 A_2) \simeq C_2 \times C_2$. Its intersection with $A_1 A_2$ must therefore be invariant under the two independent inversions on $A_1 A_2 \simeq \mathbb{F}_3^2$. The only invariant lines are the two coordinate axes. Hence H contains A_1 or A_2 , again giving $T\beta = 0$.

Thus T annihilates every element of L of degree at most five, and therefore every sum of such elements. It does not annihilate ab . Theorem 7.1 extends the obstruction to summands outside L . We have proved

$$D_+(ab) = 6, \quad D_+^{\mathbb{Q}(ab)}(ab) = 9.$$

The normal-closure step is consequently essential in general, not merely an overly cautious implementation choice.

9 What can be asserted for unrestricted products?

The exact two-factor criterion and the global prime-divisor bound already solve the product example completely. They should not be promoted, without another theorem, to a general arbitrary-product algorithm.

There is a tempting but invalid shortcut using norms. If a factorization is defined in a finite Galois extension $M/\mathbb{Q}$, and $L/\mathbb{Q}$ is a Galois subextension containing α , then

$$\alpha^{[M:L]} = \prod_i N_{M/L}(\beta_i).$$

Each norm belongs to L . Moreover, by the same equivariance argument as for trace, its absolute degree is at most that of β_i . But this proves a statement about a *power* of α , not about α itself. Extracting roots can increase absolute degrees and introduces root-of-unity ambiguities. For example, $\sqrt[3]{2}^3 = 2$ is rational, but $\sqrt[3]{2}$ is not a product of quadratic numbers by Theorem 2.1. Merely testing membership of a power in a low-degree multiplicative subgroup is therefore insufficient.

9.1 An exact bounded search for any number of factors

Fix degree, coefficient-height, and component-count bounds d, H, R . Enumerate every primitive integer polynomial of degree at most d , positive leading coefficient, and coefficients of absolute value at most H . Keep the irreducible polynomials and enumerate all their exact roots. This is a finite catalog containing precisely the possible components within that polynomial-height convention. Enumerate tuples of length at most R and check their products by **RootReduce**. The same construction works for sums. A finite search over this entire box can determine its minimum largest degree as well, by examining bounds in increasing order.

The package includes **FiniteCatalogSearch** for this last stage. Repetitions are allowed, and all successes are exact certificates. Expanding H and R in a fair order eventually finds any existing factorization at a specified bound d . This is a positive semidecision procedure: failure at all stages cannot be recognized simply by waiting for the search to finish. In particular, one must not search $d = 1, 2, 3, \dots$ sequentially with an unbounded search at each d and claim to obtain an optimizer.

For production work, a useful division of labor is to run field-based searches, small-height searches, and educated resultant ansatzes as proposal mechanisms, while a separate exact checker verifies identities, degrees, and the scope of any optimality assertion. Ideal, unit, and multiplicative-relation methods may also be useful inside a fixed field, but require their own completeness proofs.

10 Using the supplied implementation

The archive contains a loadable package rather than notebook-specific cells. A basic session is:

```
Get["RootDecomposition.wl"];

productResult = MinTwoFactorInField[alphaProduct];
sumResult = MinSumGlobally[alphaSum];

productResult["Components"]
sumResult["Components"]
{productResult["MaxDegree"], sumResult["MaxDegree"]}
(* {3, 3} *)
```

For another ambient field, pass its generator explicitly:

```
MinSumInField[alpha, "AmbientGenerator" -> theta]
MinTwoFactorInField[alpha, "AmbientGenerator" -> theta]
```

The code checks that the input belongs to that field. It returns associations containing the components, their individual degrees, an inactive expression, an exact-identity flag, a scope description, and the global prime-divisor lower bound. **GlobalMinimumCertified** is true only when the witness meets that bound or the complete normal-closure additive procedure establishes the optimum. A **Failure** describing a resource limit is not a decomposition and is not a proof of impossibility.

Entry point	Guarantee when it succeeds
<code>PairFromPolynomials</code>	Exact branch-certified witness for the specified polynomials.
<code>BoundedTwoRootSearch</code>	Exact witness in a bounded first-polynomial search; no general negative certificate.
<code>AllSubfieldBases</code>	Complete list of embedded subfields, unless an explicit failure is returned.
<code>MinSumInField</code>	Minimum for all finite sums in the specified field.
<code>MinTwoFactorInField</code>	Minimum for at most two factors in the specified field.
<code>MinSumGlobally</code>	Unrestricted additive minimum, when completed.
<code>FiniteCatalogSearch</code>	Exact witness using a finite catalog and bounded component count.

10.1 Validation status

The Wolfram computation service available during preparation returned HTTP 404 errors, and no local Wolfram kernel was installed. Consequently this article does not claim that `RootDecomposition.wl` or `tests.wlt` was executed in Mathematica. The Wolfram implementation was checked for balanced syntax and reviewed against the cited documented primitives.

Independent exact implementations in SymPy *were* executed. They verified: the two degree-nine resultants, irreducibility and real-root counts; the 1, 2, 2, 4 factor-degree patterns over the input fields; the complete subfield degree lists 1, 3, 3, 9; the additive row-space and multiplicative nullspace algorithms; the internal minima summarized below; and the sextic matching-sum elimination, irreducibility, and real-root count. Additional independent checks cover the degree-eight three-factor example and the degree-six counterexample to degree-product divisibility. The tests and logs are included rather than being reported as native Mathematica output.

Input	Degree	Internal additive minimum	Internal two-factor minimum
ab	9	9	3
$a + b$	9	3	9

The second row's two-factor value is not an assertion about arbitrary-length products outside the input field. Separately proved global results are $D_{\times}(ab) = 3$, $D_{+}(a + b) = 3$, and $D_{+}(ab) = 6$.

10.2 Reproducibility

Run the Wolfram test file with

```
TestReport["tests.wlt"]
```

from the extracted directory. The independent checks can be rerun with

```
python verify.py
python verify_algorithms.py
```

They require SymPy. Compile the article from the same directory using

```
pdflatex article.tex && pdflatex article.tex
```

The full package is typeset in the appendix from the same `.wl` file that is distributed, avoiding two diverging copies of the implementation.

11 Conclusions

For the two examples, the implementation can discover the relevant cubic subfields automatically and certify decompositions of largest degree three. A prime-divisor argument then proves that no lower degree is possible, even with arbitrarily many components.

The broader additive problem has a clean exact resolution: normalized trace reduces it to a finite Galois field, and low-degree subfield spans turn it into rational linear algebra. The multiplicative two-factor problem has a similarly clean local criterion, $E \cap \alpha F \neq 0$, but arbitrary-length products are a different optimization problem. Keeping those scopes explicit is what turns a plausible simplification routine into a mathematically reliable tool.

References

- [1] Vladimir Reshetnikov, “Factor a polynomial Root into Roots of smallest possible degree,” Mathematica Stack Exchange, question 105933 (2016), with answers by yarchik and Daniel Lichtblau (2019). <https://mathematica.stackexchange.com/questions/105933/>.
- [2] Wolfram Research, *Root*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Root.html>.
- [3] Wolfram Research, *RootReduce*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [4] Wolfram Research, *MinimalPolynomial*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/MinimalPolynomial.html>.
- [5] Wolfram Research, *Resultant*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Resultant.html>.
- [6] Wolfram Research, *Factor*, including algebraic extension options, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Factor.html>.
- [7] Wolfram Research, *ToNumberField*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToNumberField.html>.
- [8] Wolfram Research, *AlgebraicNumber*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/AlgebraicNumber.html>.
- [9] Wolfram Research, *Solve*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Solve.html>.
- [10] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022. In particular Sections 3 and 5. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [11] Andreas-Stephan Elsenhans and Jürgen Klüners, “Computing subfields of number fields and applications to Galois group computations,” preprint arXiv:1610.06837 (2016). <https://arxiv.org/abs/1610.06837>.
- [12] The PARI Group, *Catalogue of GP/PARI Functions: General Number Fields*, entry `nfsubfields`. https://pari.math.u-bordeaux.fr/dochtml/html-stable/General_number_fields.html.

A Complete Wolfram Language reference implementation

The code deliberately favors an inspectable exact algorithm over asymptotically optimal subfield enumeration. It uses rational-polynomial `Root` objects, rejects inexact input, and reports search limits explicitly.

```
(* RootDecomposition.wl -- exact arithmetic reference implementation.
See article.pdf for proofs and, especially, the scopes of minimality.
No call in this file performs an unrestricted rational-point search.
A resource-limit Failure is NOT a proof of nonexistence.
Native Wolfram-kernel execution was unavailable during preparation;
the underlying algorithms were independently exercised in SymPy. *)

BeginPackage["RootDecomposition`"];
AlgebraicDegree::usage = "AlgebraicDegree[a] gives [Q(a):Q] for an exact algebraic number.";
PrimeDegreeLowerBound::usage = "PrimeDegreeLowerBound[a] is a global lower bound for both decomposition
problems.";
ComposedPolynomial::usage = "ComposedPolynomial[p,q,x,\"Sum\"|\"Product\"] gives the monic composed polynomial.
";
ExactRootList::usage = "ExactRootList[p,x] lists all distinct roots of a rational polynomial exactly.";
PairFromPolynomials::usage = "PairFromPolynomials[a,p,q,x,operation] certifies a root pair, including its
branches.";
PairFromOnePolynomial::usage = "PairFromOnePolynomial[a,p,x,operation,d] tests every root b of p and the
complementary term/factor.";
BoundedTwoRootSearch::usage = "BoundedTwoRootSearch[a,operation,d,h] searches first-component minimal
polynomials of degree <= d and primitive integer coefficient height <= h.";
AllSubfieldBases::usage = "AllSubfieldBases[theta] returns all embedded subfields of Q(theta) as rational row
bases, or an explicit Failure.";
MinSumInField::usage = "MinSumInField[a] minimizes the largest degree for any finite sum inside Q(a). Option \
"AmbientGenerator\" specifies a different ambient field.";
MinTwoFactorInField::usage = "MinTwoFactorInField[a] minimizes the largest degree for a product of at most two
factors inside Q(a). It does not minimize arbitrary-length products.";
MinSumGlobally::usage = "MinSumGlobally[a] solves the unrestricted additive problem by a normal-closure
reduction, subject to explicit resource limits.";
FiniteCatalogSearch::usage = "FiniteCatalogSearch[a,catalog,operation,r] searches products/sums of at most r
members of a finite exact catalog.";

Begin["`Private`"];
$failureTag = "RootDecompositionFailure";
SetAttributes[guarded, HoldAll];
guarded[body_] := Catch[body, $failureTag];
require[condition_, tag_, data_ :<|>] :=
  If[! TrueQ[condition], Throw[Failure[tag, data], $failureTag]];
need[x_] := If[FailureQ[x], Throw[x, $failureTag], x];
ratQ[x_] := IntegerQ[x] || Head[x] === Rational;
zeroQ[x_] := TrueQ[Quiet[Check[RootReduce[x], $Failed]] === 0];

exactNumber[a_] := Module[{b},
  require[FreeQ[a, _Real], "InexactInput", <|"Input" -> a|>];
  b = Quiet[Check[RootReduce[a], $Failed]];
  require[b != $Failed && FreeQ[b, _Real], "InvalidAlgebraicInput"];
  b
];
monic[p_, x_Symbol] := Module[{f = Expand[p], n, c},
  require[PolynomialQ[f, x], "NotPolynomial"];
  n = Exponent[f, x];
  require[IntegerQ[n] && n >= 1, "NonpositiveDegree"];
  c = CoefficientList[f, x];
  require[VectorQ[c, ratQ], "NonrationalCoefficients"];
  Expand[f/Last[c]]
];
minpoly[a_, x_Symbol] := Module[{p},
  p = Quiet[Check[MinimalPolynomial[exactNumber[a], x], $Failed]];
  require[p != $Failed, "MinimalPolynomialFailed"];
  monic[p, x]
];
```

```

degree[a_] := Module[{x = Unique["x"]}, Exponent[Minpoly[a, x], x]];
AlgebraicDegree[a_] := guarded[degree[a]];
lowerBound[a_] := Module[{n = degree[a]},
  If[n == 1, 1, Max[FactorInteger[n][[All, 1]]]]
];
PrimeDegreeLowerBound[a_] := guarded[lowerBound[a]];
opHead[op_] := Switch[op, "Sum", Plus, "Product", Times,
  _, require[False, "UnknownOperation", <|"Operation" -> op|>]];

ComposedPolynomial[p_, q_, x_Symbol, op_] := guarded[Module[
  {f = monic[p, x], g = monic[q, x], t = Unique["t"], m, c, h},
  m = Exponent[g, x]; c = CoefficientList[g, x];
  h = Switch[op,
    "Sum", Expand[g /. x -> x - t],
    "Product", Sum[c[[j + 1]] x^j t^(m - j), {j, 0, m}],
    _, require[False, "UnknownOperation"]
  ];
  monic[Resultant[f /. x -> t, h, t], x]
]];

ExactRootList[p_, x_Symbol] := guarded[Module[{f = monic[p, x], fun},
  f = PolynomialQuotient[f, PolynomialGCD[f, D[f, x]], x];
  fun = Function[Evaluate[f /. x -> Slot[1]]];
  Table[Root[fun, j], {j, Exponent[f, x]}]
]];

certificate[a_, pieces_List, op_, scope_] := Module[{d, h = opHead[op]},
  require[Length[pieces] >= 1, "EmptyCertificate"];
  require[zeroQ[(h @@ pieces) - a], "IdentityCertificationFailed"];
  d = Max[degree /@ pieces];
  <|"Components" -> pieces,
    "InactiveExpression" -> Switch[op, "Sum", Apply[Inactive[Plus], pieces],
      "Product", Apply[Inactive[Times], pieces]],
    "ComponentDegrees" -> (degree /@ pieces),
    "MaxDegree" -> d,
    "ExactIdentityVerified" -> True,
    "Scope" -> scope,
    "GlobalLowerBound" -> lowerBound[a],
    "GlobalMinimumCertified" -> (d == lowerBound[a])|>
  ];

PairFromPolynomials[a_, p_, q_, x_Symbol, op_] := guarded[Module[
  {aa = exactNumber[a], h = opHead[op], r, pairs, hit},
  r = need[ComposedPolynomial[p, q, x, op]];
  require[Expand[PolynomialRemainder[r, minpoly[aa, x], x]] == 0,
    "NoPairForThesePolynomials"];
  pairs = Tuples[{need[ExactRootList[p, x]], need[ExactRootList[q, x]]}];
  hit = SelectFirst[pairs, zeroQ[(h @@ #) - aa] &, Missing["NotFound"]];
  require[! MissingQ[hit], "BranchSearchFailed"];
  certificate[aa, hit, op, "Specified polynomial pair"]
]];

PairFromOnePolynomial[a_, p_, x_Symbol, op_, d_Integer?Positive] := guarded[
  Module[{aa = exactNumber[a], roots, b, c},
    opHead[op]; roots = need[ExactRootList[p, x]];
    Do[
      b = roots[[j]];
      If[degree[b] > d || (op == "Product" && zeroQ[b]), Continue[]];
      c = RootReduce[If[op == "Sum", aa - b, aa/b]];
      If[degree[c] <= d,
        Return[certificate[aa, {b, c}, op, "One specified polynomial"]],
        {j, Length[roots]}];
    Failure["NoPairForThisPolynomial", <|"DegreeBound" -> d|>]
  ];
];

Options[BoundedTwoRootSearch] = {"MaxCoefficientVectors" -> 100000};
BoundedTwoRootSearch[a_, op_, d_Integer?Positive, h_Integer?Positive,
  OptionsPattern[]] := guarded[Module[
  {aa = exactNumber[a], x = Unique["x"], count, checked = 0,
    cs, p, result, top, tuple, lo},
    opHead[op]; lo = lowerBound[aa];
    require[d >= lo, "ImpossibleDegreeBound",

```

```

    <|"ProvedGlobalLowerBound" -> 1o|>];
If[degree[aa] <= d,
  Return[certificate[aa, {aa}, op, "One-component solution"]];
count = Sum[h (2 h + 1)^m, {m, 2, d}];
require[count <= OptionValue["MaxCoefficientVectors"], "ResourceLimit",
  <|"CoefficientVectorsRequired" -> count|>];
Do[
  Do[
    cs = Append[tuple, top];
    If[Apply[GCD, Abs[cs]] != 1, Continue[]];
    p = cs . x^Range[0, m];
    If[! TrueQ[IrreduciblePolynomialQ[p]], Continue[]];
    checked++;
    result = PairFromOnePolynomial[aa, p, x, op, d];
    If[FailureQ[result] && result[[1]] != "NoPairForThisPolynomial",
      need[result]];
    If[! FailureQ[result],
      Return[Join[result, <|"FirstPolynomial" -> p,
        "PolynomialVariable" -> x,
        "CheckedPolynomials" -> checked|>]],
    {top, 1, h}, {tuple, Tuples[Range[-h, h], m]}],
    {m, 2, d}];
Failure["NoPairInHeightBound", <|"DegreeBound" -> d,
  "FirstPolynomialHeightBound" -> h,
  "CheckedPolynomials" -> checked,
  "GlobalNonexistenceProved" -> False|>]
];

(* A single, integral ambient generator prevents silent basis changes. *)
fieldData[a_] := Module[{theta = exactNumber[a], x = Unique["z"], p, s, n},
  p = minpoly[theta, x];
  s = Apply[LCM, Denominator /@ CoefficientList[p, x]];
  theta = RootReduce[s theta]; p = minpoly[theta, x];
  n = Exponent[p, x];
  require[VectorQ[CoefficientList[p, x], IntegerQ], "IntegralGeneratorFailed"];
  <|"Generator" -> theta, "Variable" -> x, "Polynomial" -> p,
    "Degree" -> n, "One" -> UnitVector[n, 1]|>
];
coordinates[b_, f_Association] := Module[{r, n = f["Degree"], v},
  r = exactNumber[b];
  If[ratQ[r], Return[PadRight[{r}, n]]];
  r = Quiet[Check[ToNumberField[r, f["Generator"]], $Failed]];
  require[Head[r] === AlgebraicNumber, "NotInAmbientField", <|"Element" -> b|>];
  require[zeroQ[r[[1]] - f["Generator"]], "UnexpectedGenerator"];
  v = PadRight[r[[2]], n];
  require[Length[v] == n && VectorQ[v, ratQ], "InvalidCoordinates"];
  v
];
fromCoordinates[v_List, f_Association] :=
  RootReduce[AlgebraicNumber[f["Generator"], v]];
fieldMultiply[u_List, v_List, f_Association] := Module[
  {x = f["Variable"], n = f["Degree"], powers, p},
  powers = x^Range[0, n - 1];
  p = PolynomialRemainder[Expand[(u . powers) (v . powers)], f["Polynomial"], x];
  PadRight[CoefficientList[p, x], n]
];
rowBasis[m_List] := Select[RowReduce[m], ! AllTrue[#, (# == 0) &] &];
algebraBasis[generators_List, f_Association] := Module[{b = {f["One"]}, c},
  While[True,
    c = rowBasis[Join[b, Flatten[Table[fieldMultiply[u, v, f],
      {u, b}, {v, generators}], 1]]];
    If[Length[c] == Length[b], Return[c]];
    b = c
  ]
];
subfieldBases[f_Association, cap_] := Module[
  {t = Unique["t"], x = f["Variable"], n = f["Degree"], fac,
    polys, linear, rest, count, subset, g, k, gens, b, out = {}},

```

```

If[n == 1, Return[{{1}}]];
fac = Quiet[Check[FactorList[f["Polynomial"] /. x -> t,
  Extension -> f["Generator"]], $Failed]];
require[ListQ[fac] && Length[fac] >= 2, "NumberFieldFactorizationFailed"];
require[AllTrue[Rest[fac], Last[#] == 1 &], "UnexpectedRepeatedFactor"];
polys = (Expand[#[[1]]/Coefficient[#[[1]], t, Exponent[#[[1]], t]] &) /@ Rest[fac];
require[AllTrue[CoefficientList[Expand[Times @@ polys -
  (f["Polynomial"] /. x -> t)], t], zeroQ], "FactorizationCheckFailed"];
linear = Select[Range[Length[polys]],
  Exponent[polys[[#]], t] == 1 && zeroQ[polys[[#]] /. t -> f["Generator"]] &];
require[Length[linear] == 1, "DistinguishedLinearFactorMissing"];
rest = Delete[polys, {First[linear]}]; count = 2^Length[rest];
require[count <= cap, "ResourceLimit", <|"DivisorsRequired" -> count,
  "AmbientDegree" -> n, "NoNonexistenceConclusion" -> True|>];
Do[
  subset = If[rest === {}, {}, Pick[rest, IntegerDigits[mask, 2, Length[rest]], 1]];
  g = Expand[polys[[First[linear]]] (Times @@ subset)];
  k = Exponent[g, t];
  If[Mod[n, k] != 0, Continue[]];
  gens = coordinates[#, f] & /@ Most[CoefficientList[g, t]];
  b = algebraBasis[gens, f];
  If[Length[b] == n/k && ! MemberQ[out, b], AppendTo[out, b],
    {mask, 0, count - 1}];
  require[MemberQ[out, {f["One"]}]] && MemberQ[out, IdentityMatrix[n]],
    "SubfieldEnumerationCheckFailed"];
  SortBy[out, Length]
];
Options[AllSubfieldBases] = {"MaxDivisors" -> 65536};
AllSubfieldBases[theta_, OptionsPattern[]] := guarded[Module[{f, b},
  f = fieldData[theta]; b = subfieldBases[f, OptionValue["MaxDivisors"]];
  <|"AmbientField" -> f, "Bases" -> b, "Degrees" -> (Length /@ b),
  "Complete" -> True|>
]];

meanTrace[a_] := Module[{x = Unique["x"], p, n},
  p = minpoly[a, x]; n = Exponent[p, x];
  -Coefficient[p, x, n - 1]/n
];

centerSum[terms_List] := Module[{means, centered, q},
  means = meanTrace /@ terms;
  centered = MapThread[RootReduce[#1 - #2] &, {terms, means}];
  q = Total[means];
  centered = DeleteCases[centered, 0];
  If[q != 0, AppendTo[centered, q]];
  If[centered === {}, {0}, centered]
];

sumInField[a_, f_Association, bases_List] := Module[
  {target = coordinates[a, f], chosen, rows, w, terms, cursor, piece, ds},
  ds = Union[Length /@ bases];
  Do[
    chosen = Select[bases, Length[#] <= d &]; rows = Join @@ chosen;
    If[MatrixRank[Append[rows, target]] > MatrixRank[rows], Continue[]];
    w = Quiet[Check[LinearSolve[Transpose[rows], target], $Failed]];
    require[ListQ[w] && VectorQ[w, ratQ], "RationalLinearSolveFailed"];
    require[Transpose[rows] . w === target, "LinearSolveCheckFailed"];
    cursor = 1;
    terms = Table[
      piece = Take[w, {cursor, cursor + Length[b] - 1}] . b;
      cursor += Length[b]; fromCoordinates[piece, f], {b, chosen};
    terms = centerSum[terms];
    Return[Join[certificate[a, terms, "Sum", "All finite sums in the specified ambient field"],
      <|"MinimalWithinScope" -> True, "AmbientGenerator" -> f["Generator"],
      "AmbientDegree" -> f["Degree"], "SubfieldDegreeThreshold" -> d|>]],
      {d, ds}];
  require[False, "InternalAdditiveSearchFailed"]
];
Options[MinSumInField] = {"AmbientGenerator" -> Automatic, "MaxDivisors" -> 65536};
MinSumInField[a_, OptionsPattern[]] := guarded[Module[{aa, theta, f, b},

```

```

aa = exactNumber[a]; theta = OptionValue["AmbientGenerator"];
f = fieldData[If[theta === Automatic, aa, theta]];
coordinates[aa, f]; b = subfieldBases[f, OptionValue["MaxDivisors"]];
sumInField[aa, f, b]
];
Options[MinTwoFactorInField] = Options[MinSumInField];
MinTwoFactorInField[a_, OptionsPattern[]] := guarded[Module[
{aa, theta, f, bases, target, pairs, e, g, scaled, ns, w, u, v, pieces},
aa = exactNumber[a]; theta = OptionValue["AmbientGenerator"];
f = fieldData[If[theta === Automatic, aa, theta]];
target = coordinates[aa, f];
If[zeroQ[aa], Return[Join[certificate[aa, {0}, "Product", "Zero input"],
<|"MinimalWithinScope" -> True|>]]];
bases = subfieldBases[f, OptionValue["MaxDivisors"]];
pairs = Flatten[Table[{i, j}, {i, Length[bases]}, {j, i, Length[bases]}, 1];
pairs = SortBy[pairs, Max[Length[bases[[#[[1]]]]], Length[bases[[#[[2]]]]]] &];
Do[
e = bases[[pair[[1]]]]; g = bases[[pair[[2]]]];
scaled = fieldMultiply[target, #, f] & /@ g;
ns = NullSpace[Transpose[Join[e, -scaled]]];
If[ns === {}, Continue[]];
w = First[ns]; u = Take[w, Length[e]] . e; v = Drop[w, Length[e]] . g;
require[! AllTrue[v, (# === 0) &], "ZeroDenominatorInIntersection"];
pieces = {fromCoordinates[u, f], RootReduce[1/fromCoordinates[v, f]]};
Return[Join[certificate[aa, pieces, "Product", "At most two factors in the specified ambient field"],
<|"MinimalWithinScope" -> True, "AmbientGenerator" -> f["Generator"],
"AmbientDegree" -> f["Degree"]|>]],
{pair, pairs}];
require[False, "InternalProductSearchFailed"]
];

Options[MinSumGlobally] = {"MaxDivisors" -> 65536, "MaxNormalDegree" -> 96};
MinSumGlobally[a_, OptionsPattern[]] := guarded[Module[
{aa = exactNumber[a], quick, x = Unique["x"], roots, common, generators, f, bases, answer},
quick = need[MinSumInField[aa, "MaxDivisors" -> OptionValue["MaxDivisors"]]];
If[TrueQ[quick["GlobalMinimumCertified"]],
Return[Join[quick, <|"Scope" -> "All finite sums over the algebraic numbers",
"GlobalProof" -> "Witness meets prime-divisor lower bound"|>]]];
roots = need[ExactRootList[minpoly[aa, x], x]];
common = Quiet[Check[ToNumberField[roots, All], $Failed]];
require[ListQ[common], "NormalClosureConstructionFailed"];
generators = Cases[common, AlgebraicNumber[g_, _List] :> g];
require[generators != {}, "NormalClosureGeneratorMissing"];
f = fieldData[First[generators]];
require[f["Degree"] <= OptionValue["MaxNormalDegree"], "ResourceLimit",
<|"NormalClosureDegree" -> f["Degree"], "BestInternalResult" -> quick|>];
Scan[coordinates[#, f] &, roots];
bases = subfieldBases[f, OptionValue["MaxDivisors"]];
answer = sumInField[aa, f, bases];
Join[answer, <|"Scope" -> "All finite sums over the algebraic numbers",
"GlobalMinimumCertified" -> True,
"GlobalProof" -> "Complete normal-closure subfield enumeration and normalized-trace descent"|>]
];

Options[FiniteCatalogSearch] = {"MaxTuples" -> 1000000};
FiniteCatalogSearch[a_, catalog_List, op_, r_Integer?Positive, OptionsPattern[]] := guarded[
Module[{aa = exactNumber[a], h = opHead[op], c, count, pieces, k},
c = DeleteDuplicates[exactNumber /@ catalog, zeroQ[#1 - #2] &];
Scan[degree, c];
If[op == "Product" && ! zeroQ[aa], c = Select[c, ! zeroQ[#] &]];
require[c != {}, "EmptyCatalog"];
k = Length[c]; count = Sum[k^j, {j, 1, r}];
require[count <= OptionValue["MaxTuples"], "ResourceLimit", <|"TuplesRequired" -> count|>];
Do[
Do[
pieces = c[[If[k == 1, ConstantArray[1, j],
1 + IntegerDigits[mask, k, j]]]];
If[zeroQ[(h @@ pieces) - aa],

```

```
        Return[certificate[aa, pieces, op, "Finite catalog and bounded component count"]],
        {mask, 0, k^j - 1}},
    {j, 1, r}];
Failure["NoRepresentationInCatalog", <|"MaxComponents" -> r,
    "GlobalNonexistenceProved" -> False|>]
]
];
End[];
EndPackage[];
```